ELEC5507: Error Control Coding

Assignment Report

Project Title:

Low Density Parity Check
Metropolitan Wireless Networks IEEE 802.16 (WiMAX)

Course Code: ELEC5507
Instructors:
Prof. Yonghui Li
Prof. Branka Vucetic

Group Number: Group 5
Group Members:

- 540351558 Ammaar İftikhar
- 540415838 LeQian Cheng
- 540523672 Sirui Zhou
- 540478811 Xinyi Wang
- 530381330 Yalan Shao
- 541031871 Yunhao Wang
- 540845655 Yunsheng Sun

Saturday 24th May, 2025

1 Introduction

The need to provide complex methods of channel encoding arises from the desire to achieve the Shannon channel capacity. The Shannon-Hartley theorem defines this channel capacity C as:

$$C = B \log_2(1 + \mathrm{SNR}),$$

where B is the bandwidth and SNR is the signal-to-noise ratio [1]. The Shannon capacity for a channel can only be reached when the size of the block sequence is large [2]. The transmission of data through channels can lead to errors due to noise in the channel [3]. The error in these channels can lead to bit flipping in the transmitted data, which results in errors in the transmitted data.

In order to detect or prevent the data error in the transmitted data, we must use coding techniques, such as error control coding, to transmit reliably. The use of traditional error correcting codes for large data is not feasible due to both higher encoding and decoding complexity. The decoding complexity of some algebraic block codes can grow exponentially with the block length, making them impractical for very large blocks. This gives rise to the need for more efficiently tailored techniques for transmitting large data sequences. Turbo Codes and LDPC are two Forward Error Correction (FEC) techniques developed for this purpose. In the case of large codes, the problem of effectively decoding the transmitted code is more difficult than the task of encoding [2].

LDPC codes were first discovered by Robert Gallagher at MIT in 1960. Due to the limitations of the hardware at that time, using LDPC codes proved to be infeasible. However, due to improvements in technology, the codes were rediscovered in 1996 [4]. LDPC codes are currently considered the state-of-the-art codes, and substantial improvements from these codes are not expected. LDPCs are now used in various standards such as 5G NR, Wi-Fi (802.11n/ac/ax), DVB-S2 (satellite television), and modern data storage devices.

	Hamming	Reed-Solomon	BCH	LDPC
Error Correction	Single-bit	Single-bit (Can detect two-bit)	Multi-bit	
Flash Type	SLC	MLC	MLC/TLC	
Error Type	Scattered	Burst	Scattered	
Soft Bit Decoding	No			Yes
Error Correction Capability	Limited	High		
Performance	Medium		High	Very High

Figure 1: Comparison of Different Error Correcting Codes [4].

Low Density Parity Codes (LDPC) have the ability to efficiently decode and encode large blocks of data that reach the Shannon capacity of a channel. LDPC codes use a sparse matrix containing mostly ones and zeros. The decoding of Shannon codes is less computationally complex in comparison to other similar performing codes: the iterative decoding of LDPC codes has linear complexity [2]. Due to the lower relative computational power

required by LDPC codes, they are preferable for higher data rates over other codes like Turbo codes [4]. The LDPC codes replaced the hard decisions in decoding with soft decisions [2]. The use of soft decisions helps in providing LDPCs with better error correcting capabilities than others.

2 Theoretical Background

LDPC codes are defined by a sparse parity-check matrix, denoted as $\mathbf{H}$. The sparsity of the parity matrix $\mathbf{H}$ is the key to their relatively low decoding complexity when compared to other high-performing codes. The low sparsity is the reason why these codes are known as low density. The LDPC codes are linear block codes typically used over binary fields.

The (n, k) binary block codes have k bits of information and $(n - k)$ bits of redundant information. The parity-check matrix $\mathbf{H}$ is of $(n - k) \times n$ dimensions. The LDPC codes specifically are represented by (n, λ, ρ) where n is the length of the codeword and λ and ρ represent the column and row weights respectively. The parameters λ and ρ are significantly smaller than n. One of the key properties of a parity-check matrix is its sparsity, represented by $r = \rho/n$.

LDPC codes are also represented by bipartite graphs called Tanner Graphs. The two sets of edges in the Tanner Graph are referred to as variable nodes and check nodes. The variable nodes represent the codeword and the check nodes represent the parity matrix rows (or equations). The edge between a variable node v and check node c exists if $h(v, c) = 1$ in the parity-check matrix $\mathbf{H}$, i.e., 1 if the v-th bit participates in the c-th equation. Higher sparsity implies fewer edges in the graph. Tanner graphs are essential in order to understand the decoding done by LDPC codes. They are also extensively used in many LDPC algorithms, like sum-product algorithm (belief propagation) decoding.

There are cases where all the columns and rows of the parity-check matrix $\mathbf{H}$ don't have the same weight; such types of LDPC codes are called irregular LDPC codes. To represent this variability LDPC codes have c-node and v-node degree distributions. The two distributions are represented by $\lambda(x)$ and $\rho(x)$.

Bit Error Rate (BER) and Frame Error Rate (FER) are two of the metrics used to compare the performance of different error correcting codes. The BER is the ratio of the number of erroneous bits to the number of total bits transmitted. On the other hand, FER represents the number of erroneous frames to the total number of frames transmitted. The BER and FER are related, and this relation depends on the frame size and existence of burst errors. The LDPC codes attempt to minimize the FER and BER at any SNR. These codes have the best performance when the size of the transmitted data is large as mentioned in the introduction.

3 Design and Implementation

3.1 System Architecture

The LDPC simulation system project in this report complies with the IEEE 802.16e (WiMAX) standard and uses a low-density parity check (LDPC) code with a code length of $N = 2304$, an information length of $K = 1152$, and a code rate of $R = 1/2$. This report project is run using MATLAB and has multiple modular functions, each of which represents a key stage in a digital communication link.

The complete data flow of this project contains six main parts: random message generation, LDPC encoding, BPSK modulation, channel simulation (AWGN or Rayleigh channel simulation), LDPC decoding based on the sum-product algorithm (SPA), and BER and FER performance evaluation.

To support this structure, the MATLAB implementation is divided into modular functions as shown in Table 1, where each function corresponds to a key stage in the system pipeline.

Table 1: Modular Functions in LDPC Simulation System

Function	**Description**
`Hbm`	Constructs the WiMAX LDPC base matrix (12×24).
`expand_Hbm`	Expands the base matrix into full-size parity-check matrix H, and splits it into H_p and H_s.
`H2P`	Converts H into systematic form $[I \mid P]$ and records column permutation history.
`encoding`	Generates encoded codeword $x = [p, s]$ based on H_s and H_p.
`reorder_bits`	Reverses column permutations to restore original bit positions.
`LDPC_SPA`	Implements iterative decoding using the Sum-Product Algorithm (SPA).
`extract_mesg`	Extracts the information bits s from the decoded codeword.
`main_AWGN`	Runs LDPC simulation over AWGN channel and outputs BER/FER results.
`main_Rayleigh`	Simulates LDPC decoding performance over Rayleigh fading channel.

3.2 Encoding Process

In the conceptual setting of LDPC coding, the goal is to transform a binary input vector into a valid codeword that satisfies a set of parity-check equations. These equations are defined by a sparse binary matrix H. In this project, we adopt the LDPC implementation under the IEEE 802.16e (WiMAX) standard, where the parity-check matrix H is

generated from a smaller base matrix H_{bm} of size 12×24. Through a process known as *lifting* with factor $z = 96$, each entry of H_{bm} is replaced with a $z \times z$ matrix according to the following rule:

- If the entry is -1: replace with a $z \times z$ all-zero matrix.
- If the entry is 0: replace with a $z \times z$ identity matrix.
- If the entry is $p > 0$: replace with a $z \times z$ identity matrix right-cyclically shifted by p positions.

The MATLAB script `expand_Hbm.m` implements this logic and produces the full binary matrix H of size 1152×2304. The resulting matrix is sparse and well-suited for hardware acceleration and efficient software processing.

However, to enable systematic encoding, matrix H must first be transformed into a more structured form using Gaussian elimination over F_2. The target format is:

$$H \to \begin{bmatrix} I_{n-k} & P^\top \end{bmatrix}$$

where I_{n-k} is an identity matrix of size $n - k$ and $P^\top$ is the transposed parity matrix. The script `H2P.m` carries out this elimination and records column permutations needed to later reverse the bit rearrangement.

Based on the systematic form $H = [I_{n-k}\ P^\top]$, the parity component P can be extracted and used to compute the parity bits from the message vector. In our implementation, the MATLAB script `encoding.m` performs this operation using a recursive bitwise algorithm derived from Algorithm 2 of the WiMAX LDPC encoder specification. The implementation logic of this step is illustrated in Figure 2, which shows the actual MATLAB code segment used to recursively compute the parity vector based on Algorithm 2.

```
% Step 2: compute parity bits p using Algorithm 2
p(1) = mod(w(1), 2);
idx = 1;
for i = 1:mb*z
    if idx > (mb-1)*z && idx <= mb*z-1
        p(idx - (mb-1)*z + 1) = mod(w(idx - (mb-1)*z + 1) + p(idx), 2);
        idx = idx - (mb-1)*z + 1;
    elseif idx > 0 && idx <= (mb-1)*z
        p(z + idx) = mod(w(z + idx) + p(idx), 2);
        idx = idx + z;
    end
end
p(mb*z) = mod(w(i) + p((mb-1)*z), 2);
```

Figure 2: MATLAB implementation of parity computation (Algorithm 2)

The encoding procedure works as follows:

1. Compute the intermediate result $w = s \cdot H_s^\top$.
2. Recursively derive the parity vector p from w using bitwise propagation.
3. Concatenate p and s to obtain the codeword:

$$x = [p\ s]$$

Finally, since column permutations were introduced during the matrix transformation in `H2P.m`, the codeword must be reordered using the inverse permutation. This is handled by the `reorder_bits.m` script, which ensures the output aligns with the systematic encoding format expected by the decoder.

The overall encoding process is illustrated in Figure 3.

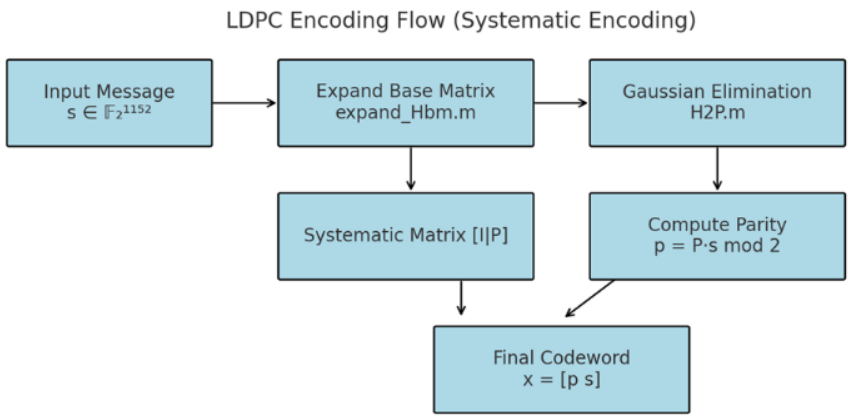

Figure 3: LDPC Encoding Flowchart (Systematic Encoding)

3.3 Decoding Algorithm (Sum-Product Algorithm - SPA)

In our project, the Sum-Product Algorithm (SPA) was used to handle the LDPC decoding process, as it was a mandatory requirement specified by the assignment. This algorithm performs very well in terms of error correction performance—especially compared to other iterative decoding methods. SPA is based on the Tanner graph structure and uses the log-likelihood ratio (LLR) message passing mechanism for probabilistic inference. Compared with bit-flipping or large number logic decoders, SPA's decoding performance for large-scale sparse codes (especially in AWGN channel conditions) is close to the channel capacity limit.

This Tanner graph is constructed based on the check matrix H, and contains two types of nodes:

- **Variable nodes**: map the encoded transmission bits (corresponding to the columns of the check matrix)
- **Check nodes**: perform parity check constraint operations (corresponding to the rows of the check matrix)

In our implementation, the graph includes 2304 variable nodes and 1152 check nodes, which correspond to the number of columns and rows of the parity-check matrix H, respectively.

Whenever a non-zero element appears in the matrix H, a corresponding edge connection is established in the graph. During the decoding process, messages are continuously passed between the two types of nodes (these messages contain not only numerical information, but also soft decision information in the form of LLR) until the algorithm converges. This soft information essentially reflects the probability likelihood of each bit being 0 or 1 under noise interference.

So when decoding starts, what we do is look at those received things—like $r_0, r_1, r_2 \ldots$ all of them up to r_{n-1}. We set LLRs from that. But yeah, those aren't perfect. We kind of assume there's Gaussian noise in the mix, or at least something messing the values a bit. Based on that, the way they relate to the actual bits can be written like this:

$$r_i = x_i + n_i, \quad \text{where } x_i \in \{+1, -1\}, \quad n_i \sim \mathcal{N}(0, \sigma^2)$$

The initial LLR for each variable node is calculated as:

$$L(v_i) = \log\left(\frac{\Pr(v_i = 0 \mid r_i)}{\Pr(v_i = 1 \mid r_i)}\right) = \frac{2r_i}{\sigma^2}$$

These LLRs represent the belief in each bit being 0 and are propagated along all edges from variable nodes to their connected check nodes.

During each iteration, the following steps are performed:

3.3.1 Check Node Update

Each check node s_j collects incoming messages from its neighboring variable nodes v_k and computes outgoing messages using the product of tanh functions:

$$\tanh\left(\frac{L(s_j \to v_i)}{2}\right) = \prod_{k \in S_j, k \neq i} \tanh\left(\frac{L(v_k \to s_j)}{2}\right)$$

Hence, the outgoing message is given by:

$$L(s_j \to v_i) = 2 \cdot \tanh^{-1}\left(\prod_{k \neq i} \tanh\left(\frac{L(v_k \to s_j)}{2}\right)\right)$$

This non-linear transformation ensures that parity consistency is enforced among bits connected to the check node.

3.3.2 Variable Node Update

Each variable node v_i receives updated estimates from its neighboring check nodes s_j. It updates its outgoing message toward check node s_j by aggregating the incoming messages and the initial LLR:

$$L(v_i \to s_j) = L(v_i) + \sum_{k \in N(v_i), k \neq j} L(s_k \to v_i)$$

After gathering all incoming messages, a posterior LLR is computed as:

$$L_{\text{post}}(v_i) = L(v_i) + \sum_{j \in N(v_i)} L(s_j \to v_i)$$

The decision is made as: $\hat{v}_i = 0$ if $L_{\text{post}}(v_i) \geq 0$; otherwise $\hat{v}_i = 1$.

3.3.3 Stopping Criterion

The decoding process is performed iteratively (the maximum number of iterations is usually set to 30 rounds), but the iteration can be terminated early as long as the convergence condition of the check equation $H \cdot \hat{x}^T = 0$ is met (that is, all parity checks pass) - this can significantly reduce the computational overhead when the channel conditions are good.

3.3.4 Numerical Stability

Before applying nonlinear functions (such as tanh or $\tanh^{-1}$), the LLR value needs to be limited (setting ± 50 as the threshold) to prevent floating-point operation exceptions. This stability control mechanism has been integrated into the `LDPC_SPA.m` script, which can effectively maintain the numerical stability of the decoder.

Specifically, in the message update step from check nodes to variable nodes, the function applies a protective bounding for extreme cases where the computed value $\tanh^{-1}(\cdot)$ may tend to infinity. As shown in the relevant part of the code:

```
if multipleVal == inf
    Uji(j, idx(k)) = 10;
elseif multipleVal == -inf
    Uji(j, idx(k)) = -10;
else
    Uji(j, idx(k)) = multipleVal;
end
```

Here, the extreme values of $+\infty$ and $-\infty$ are clipped to $+10$ and -10, respectively, to avoid instability during further iterations. This approximation retains decoding performance while preventing overflow or NaN errors in MATLAB's floating-point environment.

The complete decoding process is implemented by the MATLAB function `LDPC_SPA.m`, and its execution steps are as follows:

- Calculate the initial log-likelihood ratio (LLR) value by receiving the signal r
- Establish a sparse message storage structure associated with the Tanner graph edge
- Iterate the message transmission between the check node and the variable node
- The iteration termination condition meets any of the following conditions:
  - The decoded bits meet all parity checks ($H \cdot \hat{x}^T = 0$)
  - Reach the maximum number of iterations (default 30 times)

It should be noted that the decoding process reuses the Tanner graph structure obtained by matrix expansion in the encoding stage. As shown in Figure 4, the figure concisely shows the message transmission path between the variable node and the check node – this iterative update mechanism is the core of the excellent error correction performance of LDPC code.

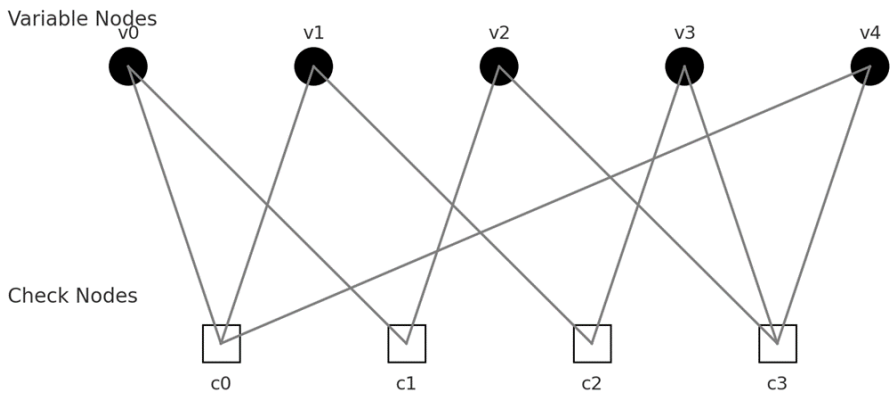

Figure 4: Tanner Graph and Message Passing in SPA

The core of SPA decoding lies in the message passing mechanism between variable nodes and check nodes. As shown in Figure 4, circles represent variable nodes, squares represent check nodes, and the connection between them is completely determined by the structure of the check matrix H. In each iteration, the updated log-likelihood ratio (LLR) value is passed along these connections to gradually optimize the bit estimation results.

Taking this specific Tanner graph as an example, each check node will impose parity check constraints on the bits it connects to. For example, the check node c_0 will impose constraints on the variable nodes v_0, v_1, v_4, requiring the condition $v_0 \oplus v_1 \oplus v_4 = 0$ to be satisfied. The message passed by the variable node to the check node reflects the likelihood

information of each bit, while the message returned by the check node is derived based on the consistency of the estimated values of adjacent bits.

Unlike directly solving large linear equations, SPA iteratively approximates the maximum a posteriori probability by performing local distributed calculations on the graph. The convergence of the decoding process is monitored by the degree of satisfaction of the check equation, and it can be intuitively observed that the message value transmitted in the network gradually tends to be stable.

3.4 Simulation Environment

The system is controlled by two main programs: `main_AWGN` and `main_Rayleigh`. In the AWGN channel simulation, the bit error rate (BER) is calculated based on a fixed number of bit errors, while in the Rayleigh channel simulation, it is calculated based on a fixed total number of transmitted codewords. Both programs generate a plot that displays the variation of bit error rate (BER) and frame error rate (FER).

4 Analysis and Comments

4.1 Base Matrix Construction and Regularity Analysis

The base matrix H_{bm} used to construct the LDPC code is a 12×24 matrix as specified in the IEEE 802.16e WiMAX standard. Each entry represents either a circulant shift value (a non-negative integer) or -1, which corresponds to a zero submatrix during expansion. The full binary parity-check matrix H is constructed by expanding this base matrix with a factor $z = 96$, resulting in a matrix of size 1152×2304.

The base matrix H_{bm} is shown below:

-1	94	73	-1	-1	-1	-1	-1	55	83	-1	-1	7	0	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	27	-1	-1	-1	22	79	9	-1	-1	-1	12	-1	0	0	-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	24	22	81	-1	33	-1	-1	-1	0	-1	-1	0	0	-1	-1	-1	-1	-1	-1	-1	-1
61	-1	47	-1	-1	-1	-1	-1	65	25	-1	-1	-1	-1	-1	0	0	-1	-1	-1	-1	-1	-1	-1
-1	-1	39	-1	-1	-1	84	-1	-1	41	72	-1	-1	-1	-1	-1	0	0	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	46	40	-1	82	-1	-1	-1	79	0	-1	-1	-1	-1	0	0	-1	-1	-1	-1	-1
-1	-1	95	53	-1	-1	-1	-1	-1	14	18	-1	-1	-1	-1	-1	-1	-1	0	0	-1	-1	-1	-1
-1	11	73	-1	-1	-1	2	-1	-1	47	-1	-1	-1	-1	-1	-1	-1	-1	-1	0	0	-1	-1	-1
12	-1	-1	-1	83	24	-1	43	-1	-1	-1	51	-1	-1	-1	-1	-1	-1	-1	-1	0	0	-1	-1
-1	-1	-1	-1	-1	94	-1	59	-1	-1	70	72	-1	-1	-1	-1	-1	-1	-1	-1	-1	0	0	-1
-1	-1	7	65	-1	-1	-1	-1	39	49	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0	0
43	-1	-1	-1	-1	66	-1	41	-1	-1	-1	26	7	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0

Definition: An LDPC code is called *regular* if every row and every column of its base matrix has the same number of non--1 entries (i.e., fixed degree). If the number of non--1 entries varies across rows or columns, the LDPC code is considered *irregular*.

Analysis:

- **Row weights:** All 12 rows contain exactly 6 non--1 entries.
- **Column weights:** The number of non--1 entries per column varies. Specifically:

$$[3,\ 4,\ 4,\ 3,\ 3,\ 5,\ 3,\ 5,\ 3,\ 4,\ 4,\ 4,\ 3,\ 2,\ 2,\ 2,\ 2,\ 2,\ 2,\ 2,\ 2,\ 2,\ 2,\ 1]$$

4.2 Simulation of the LDPC Coding System over AWGN Channel with BPSK Modulation

In our simulation, we evaluated the Bit Error Rate (BER) performance of the LDPC-coded system over an Additive White Gaussian Noise (AWGN) channel using BPSK modulation. To ensure consistent and statistically meaningful BER estimation, we adopted a fixed-error-counting method.

For each E_b/N_0 point, the simulation continues until a total of 1000 bit errors are accumulated. Once this threshold is reached, the BER is calculated as the ratio of the total number of bit errors to the total number of transmitted bits, and the simulation for that point is terminated.

To avoid excessively long simulation times when BER is low, we introduced a maximum frame limit of 10,000 frames. If this limit is reached before accumulating 1000 bit errors, the simulation is terminated early at that E_b/N_0 value.

Moreover, we implemented an early stopping mechanism to further improve simulation efficiency. If 100 consecutive frames are received without any bit errors, the simulation is halted at that SNR value, and the BER is recorded as zero, assuming a negligible error probability at that point.

This methodology ensures a balance between accuracy and computational efficiency, especially in high SNR regions where error events are rare.

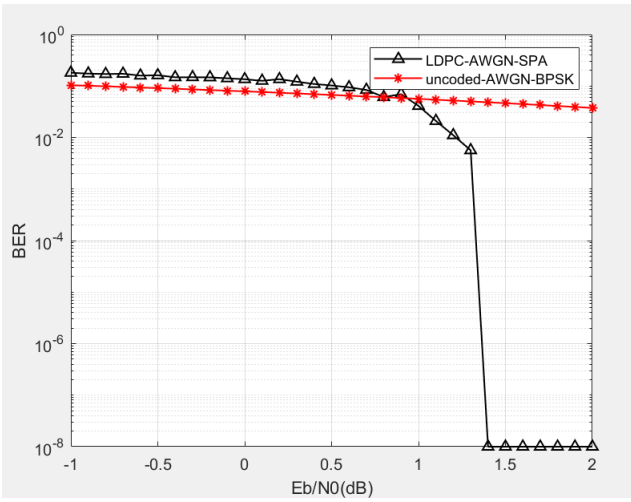

Figure 5: BER of LDPC_SPA decoding AWGN

Figure 5 shows the BER performance of the LDPC-coded system under AWGN channel using SPA decoding, compared with uncoded BPSK.

At low SNR values (below 1 dB), the LDPC code does not demonstrate a significant advantage, and the BER is even worse than that of uncoded transmission. However, as the SNR increases, the performance improves rapidly. With only about a 1.5 dB increase in E_b/N_0, the BER drops sharply to nearly zero, demonstrating the strong error correction capability of LDPC codes under higher SNR conditions.

4.3 LDPC Performance under Rayleigh Channel

In our simulation, we evaluated the Bit Error Rate (BER) performance of the LDPC-coded system over a Rayleigh fading channel using BPSK modulation. Unlike the AWGN channel which uses a fixed-error-counting strategy, the Rayleigh channel simulation adopts a fixed-bit-transmission approach. For each E_b/N_0 point, a predefined number of codewords is transmitted—500 rounds for low SNRs ($E_b/N_0 < 6$ dB) and up to 1000 rounds for higher SNRs. This ensures consistent statistical sampling across SNR levels.

To improve efficiency, we also applied the same early stopping mechanism used in the AWGN simulation: if 100 consecutive frames are decoded without any bit errors, the simulation at that E_b/N_0 is terminated and the BER is recorded as zero.

The fixed-bit method was chosen instead of the fixed-error-counting method due to the extremely low BER observed at high SNR (often around 10^{-4} to 10^{-5}), which would otherwise require simulating over 100,000 frames to observe sufficient errors—an impractical workload under Rayleigh fading with highly variable realizations.

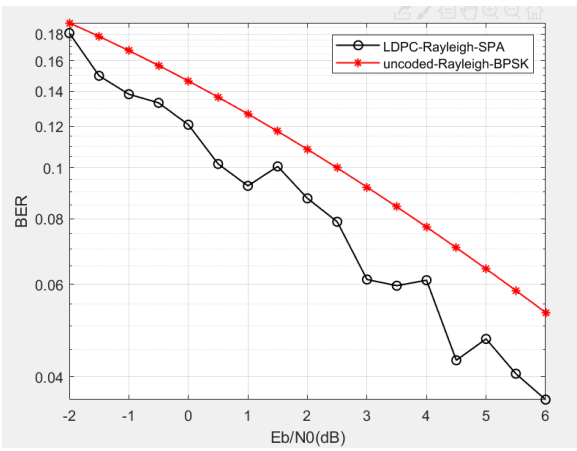

Figure 6: LDPC BER performance over Rayleigh channel from -2 dB to 6 dB

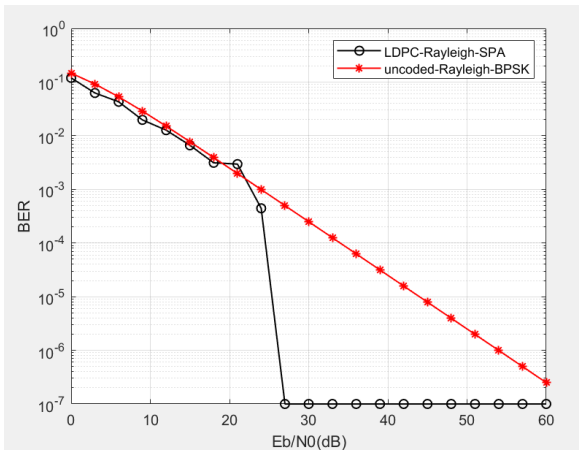

Figure 7: LDPC BER curve under Rayleigh channel over 0–60 dB showing waterfall behavior

Figure 6 shows the BER performance over a Rayleigh fading channel in the SNR range of -2 dB to 6 dB. Compared with the uncoded BPSK or traditional systems, an improvement of approximately 1–2 dB in BER performance is observed, indicating the robustness

of LDPC coding under multipath fading environments.

To further explore the behavior of LDPC decoding in extreme SNR ranges, we extended the test interval up to $E_b/N_0 = 60$ dB. As shown in Figure 7, the bit error rate drops rapidly to zero within the 20–30 dB region, forming the classic "waterfall" curve. This highlights the strong error correction capability of LDPC codes and their near-capacity decoding behavior under Rayleigh fading conditions.

5 Conclusion

This project deepened my understanding of the theoretical foundations of LDPC coding and enhanced my MATLAB programming skills through practical implementation. By designing and simulating an IEEE 802.16e-compliant LDPC system using MATLAB, I was able to explore both the conceptual aspects of error correction coding and its behavior under realistic transmission environments.

The main achievements of this project include the successful implementation of LDPC encoding and decoding modules, BPSK modulation, AWGN and Rayleigh channel modelling, and accurate BER/FER performance evaluation. Simulation results show that the LDPC code has excellent error correction performance—particularly in AWGN channels where the BER is close to the theoretical limit, and in Rayleigh fading environments where it demonstrates reliable decoding even under multipath effects. These results validate the strength of LDPC codes in both static and fading channels, showcasing their capacity-approaching and noise-resilient nature.

One of the difficulties I encountered was the complexity of matrix operations, especially when converting the parity-check matrix into systematic form for encoding. Another significant challenge arose during the simulation over Rayleigh channels: in high-SNR scenarios, the transmitted signal can produce extremely large LLR values, which in turn may lead to instability or failure of the SPA decoder. This required careful design to ensure that decoding remained accurate under such extreme conditions.

Overall, the project met all the expected requirements. It covered all the fundamental components of a digital communication system and allowed me to explore LDPC codes both theoretically and practically. If there is anything I would improve on in the future, it would be to address the security limitations of using a fixed, publicly known protograph base matrix (also referred to as the small H matrix). For applications involving secure communications, dynamically generated or randomized base matrices could enhance confidentiality and make the system more resistant to code-based attacks.

6 References

References

[1] ”Hartley Theorem - an overview,” ScienceDirect Topics. [Online]. Available: https://www.sciencedirect.com/topics/computer-science/hartley-theorem. [Accessed: 2025-05-06].

[2] R. G. Gallager, ”Low-density parity-check codes,” Massachusetts Institute of Technology, Cambridge, MA, Research Laboratory of Electronics, Technical Report 374, 1959. [Online]. Available: https://web.stanford.edu/class/ee388/papers/ldpc.pdf. [Accessed: 2025-05-06].

[3] ”Transmission Error - an overview,” ScienceDirect Topics. [Online]. Available: https://www.sciencedirect.com/topics/computer-science/transmission-error. [Accessed: 2025-05-06].

[4] D. Mackenzie, ”Communication speed nears terminal velocity,” New Scientist, vol. 187, no. 2507, pp. 30-33, Jul. 2005. [Online]. Available: https://www.newscientist.com/article/mg18725071-400-communication-speed-nears-terminal-velocity/. [Accessed: 2025-05-06].

[5] LDPC SSD: Low-Density Parity-Check ECC Algorithm,” ATP Inc. [Online]. Available: https://www.atpinc.com/blog/ldpc-ssd-low-density-parity-check-ecc-algorithm. [Accessed: 2025-05-06].

AI Usage Declaration:
We acknowledge the use of AI tools, specifically ChatGPT (GPT-4o), to assist in literature search, code verification, and checking for grammatical and spelling errors.

Group Contribution Summary

Individual Contributions of Group Members(Detailed Version)

Student Name	SID	Code	Report	Presentation
Sirui Zhou	540523672	80%	20%	–
Ammaar İftikhar	540351558	20%	20%	–
Yunhao Wang	541031871	–	30%	–
LeQian Cheng	540415838	–	30%	–
Xinyi Wang	540478811	–	–	33.3%
Yalan Shao	530381330	–	–	33.3%
Yunsheng Sun	540845655	–	–	33.3%

Normalized Group Contribution Percentage

Student Name	SID	Total Contribution (%)
Sirui Zhou	540523672	16%
Ammaar İftikhar	540351558	14%
Yunhao Wang	541031871	14%
LeQian Cheng	540415838	14%
Xinyi Wang	540478811	14%
Yalan Shao	530381330	14%
Yunsheng Sun	540845655	14%